```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        if (id % 2 == 0)
        {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
        else
        {
```

```
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);

            pthread_mutex_lock(&forks[left]); //
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }

    return NULL;
}

int main()
{
    int i;
    int ids[N];

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
```

```
    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

Output:

```
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 is thinking.
Philosopher 0 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 1 picked up right fork 2.
Philosopher 4 picked up left fork 4.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 1 is thinking.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 0 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
```